

A Report on HEARTS

1. Problem Selection

A large part of our motivation in choosing our problem was that it seemed like it would be fun and that almost everything else we could think of was either hair-tearing hard or tearfully boring. Clearly, a traditional technology *could* be used to solve this problem, and in fact the best game playing programs tend to use heuristic search methods (traditional AI) to solve such problems. The problem with using that approach in Hearts is that heuristic search only works in combinatoric spaces when you have a combination of processing muscle and good heuristics for focusing the search. This seems a very unlikely approach in Hearts on a Xerox DanddTiger. The uncertainty about opponents' hands and intentions (as well as other psychological factors) makes the space of possibilities extremely large, but worse yet, heterogenous - thus making unguided search infeasible. Guided search, on the other hand, awaits the development of powerful heuristics which was part of the goal of our project.

A careful look at the problem space, however, also raises doubts about the appropriateness of current expert system technology. In fact, had we more carefully considered whether a rule-based approach would be expressive enough for this domain, we might have moved on to some other problem. In retrospect, I'm glad we stuck with Hearts because one of my current beliefs is that the technology is adequate for making a reasonably competent hearts player, even if it isn't adequate for representing the full range of knowledge and skills used by a human player. In any case, heuristics are a critical part of any total solution and a rule-based

architecture provides a framework for formulating the necessary heuristic knowledge as well as for studying other relevant issues.

Below, I, for the most part, bypass checklisting the selection criteria, and concentrate on an account of the difficulties presented by the problem. Two aspects of our problem space make it particularly demanding. First, the problem is not really a single problem (as revealed by a lack of a complete simply-stated description) but rather a collection of problems presented by the exigencies of the game's rules. Our system represents players as objects and the demands of the game are embodied in messages sent to the player from the house (the iron-willed game administrator). This fragments the problem space into an initial collection of sub-problems, that are (and here's the rub) inter-related. This introduces at the outset a requirement to organize the rules by tasks and a departure from the cleanest model of a rule-based system.

Second, good hearts players (hereafter experts) use a wide variety of knowledge and skills. A central core which all experts are likely to use does exist; however, there are plenty of peripheral skills or infrequently applied strategies which a particular expert may not use. The problem this presents is that many of the strategies or skills which an expert may use are not easily (not to mention naturally) expressed in rules. An example makes this clear: there is a wide range of how detailed a memory of the cards played an expert may record, but let's say that the best, like our expert system, remember everything. Now, some experts will use a little probability or even search to pick a card to play. This is the kind of skill that suggests that other paradigms may be needed to make a truly expert player.

It seems reasonable, however, that the control of a multi-paradigm solution could be handled by a rule based system. A large part of game-playing is assessing situations and generating interesting and feasible tasks. In Hearts, experts use heuristics to generate tasks, and so rules are a natural representation for this skill. This has further^{1e} divisive effects on the rule base into packets responsible for situation evaluation and task performance, though we could use other types of components for the latter (e.g. a hide-and-go-seek or a gambling expert).

2. Problem Coverage

HEARTS is capable of playing entire games, though its execution speed is approaching unacceptable to be entertaining. A ready source of performance testing is provided by the dozen or so experts (and for that matter the dozens of intelligent people) we know. HEARTS isn't quite expert enough to beat a reasonably skilled player at this stage. HEARTS is capable, however, of doing all the 'interesting' things: namely, it can shoot the moon and ~~e~~clipse the moon, in addition to minimizing points taken.

HEARTS uses rules to pick cards to pass and play, and to evaluate situations to determine strategies and to model opponents. The system currently has about 90 rules split into 16 rule classes. I would guess that we could make HEARTS very good in another two weeks to a month with about twice as many rules. Further^{er} improvement in performance would require adding components which more accurately represent skills like search and planning.

3. Knowledge Formulation

We are both arguably Hearts experts (we argued many times over who was more expert), which greatly simplified the Knowledge Engineering task. An abstract model of our initial design sessions might include two experts and two knowledge engineers, but this understates the communication overhead between an expert and a knowledge engineer.

We were able to build a general framework for HEARTS in a few hours of discussion. We decided that there should be three classes of rules, two to deal with the game requirements of passing and playing, and one for evaluation of the game state to determine strategies and to model opponents. We further subdivided the passing and playing rule classes into subclasses for different strategies, and the evaluation rule class into a set of subclasses, one for each point during the play when it made sense to assess new information and review our position.

In this initial discussion, we quickly isolated five general strategies used by an expert: minimizing points taken, shooting the moon, eclipsing the moon, protecting an opponent from going over the game termination score of 100, and nailing an opponent who is winning (or maybe because you hate her). We decided not to implement the last two strategies at first, because they are infrequently used and also because it complicated matters since these strategies only operate in addition to one of the first three.

Our organization of our rule base into separate packets to handle different tasks in Hearts reminded me of R1 which performed a task which was composed of many separate but interconnected subtasks. However, in our case, because we are experts, our initial formulation was relatively painless because it was easy to compare it to an expert's sense of the structure of the domain. We probably missed out on some of the lessons to be learned by having to develop even this first model by interviewing somebody else. Certainly mapping out a set of rule classes to deal with various subproblems could be a significant effort in a domain which was foreign to the knowledge engineer.

After we laid out the rule classes, we converted the knowledge embedded in a Lisp-based player, the Conservative player, that we had originally written to test our Hearts support environment. This player has no memory of what happens, doesn't pay attention to what his opponents are up to, and blithely uses a set of rules which are a first cut at a minimizing strategy.

At this stage, we conducted a hearts session with some of our friends. During this session through a series of open hand and open mind games we discovered quite a few rules. But more interestingly, we were very discouraged by our attempts to capture some of the more interesting strategies in rules. It seemed to us that shooting the moon was largely a planning task, and that heuristics were very inadequate near the end of a hand. In addition to these two observations, we also isolated a slew of intermediate level goals and issues that played a part in the decision process of an expert. For example, trying to get or ~~lose~~^{lose} the lead. We weren't sure how to integrate all this knowledge into rules, or maybe more importantly, we weren't sure how to control the use of this knowledge. It seemed that we needed to define a large

vocabulary of new concepts, and needed to think hard about the design of layers which focussed on differing levels of detail.

We plowed ahead, however, assuming that we could go a significant distance before wearing out the framework we had already put in place. But this session did lead to a further subdivision of the playing rule classes (for each strategy) into packets to handle different types of play: leading, following suit, and dumping cards. By now, we were getting a sense of how important this hierarchy of rule classes was in the Hearts domain. We added packets of rules to the evaluation class for opponent modelling, and then we began the slow process of improving the performance of HEARTS.

After this, the structure of the program changed very little, as we improved the program by observing its performance, and adding rules to handle new cases which arose. A significant difficulty during this stage was assigning weights to rules so that they had the right firing order. Moreover the use of shared rules created 'pinned points' which imposed a partial ordering of these rules within every class they were part of. After a couple of weeks of this rule addition and improvement process, HEARTS was starting to behave interestingly, and I started feeling that heuristic rules were powerful enough to make a reasonably good player, and that, in any case, rules had been a good way of groping for the knowledge which experts use to solve several of the subtasks commonly executed in Hearts.

4. Program Architecture

As mentioned above, one of the most striking features of our rule-base is its fragmentation into hierarchically organized packets. This heirarchical organization itself and the Lisp code to control its use represents a significant amount of knowledge. In fact, selection of packets could be viewed as steps in the reasoning process (and so HEARTS exhibits chains of reasoning which are three or four levels deep). R1 also initially used Lisp code to implement the flow of control through phases. Later in its lifetime, this code was recast in rules; my first reaction to this was oh god, why? Why remove the structure in the task imposed by that lisp code?

Now I realize that the recasting didn't flatten the structure (i.e. it didn't mean that the rule packets were dissolved into one big ocean of rules), but rather it created a meta-level packet of rules which explicitly stated the control knowledge. We have highlighted this knowledge in our listing of the messages and functions of the expert player. This control knowledge could be expressed in rules, providing a fluid representation. So far, it hasn't been important in our Hearts domain, but in other domains where the structure of the rule base is more likely to change significantly as the knowledge expands, this technique would be more useful. Another important consideration is efficiency (yes, an engineering issue): the KEE Rule System is slow which is a strong argument against using rules for this purpose. We could discharge this argument by compiling the rules into an efficiently executable representation (if only we had such a compiler).

This raises the issue of where else knowledge has been encoded in Lisp and which parts of this knowledge is reasonably recast as rules or in some other form. First, the game's rules are imposed by the interface to the house and by services provided by the house (the legal cards to play). This knowledge is not important if performance is the main issue (as it tends to be in expert systems); however, as will be discussed below, it is very important if the system must learn or reason about its heuristics. Second, we provide lots of data-structures and access functions which encode the basic vocabulary for rule expression. This knowledge clearly isn't expressible in rule, but a frame language (as provided by KEE or LOOPS) seems highly appropriate. This type of representation would be very useful for writing more potent rules since using a frame-based vocabulary would buy us into the reasoning services provided by the frame interpreter.

Now, we return to the rules and their operation. The rules bind their local variables by assertions in the premise list. Other options are to use global variables and calls to eval to reference them; or not to use local variables and live with the naturally expressed but inefficient rules. Our choice (the one that KEE seems to facilitate) leads to premises which are merely functional and which consequently muddy the logic of the premise statement. In fact, it makes the use of premise complexity or specificity as methods of conflict resolutions an unworkable approach. More on this matter and other problems with KEE are discussed in Appendix 1.

In playing and passing, we run the rules ordered by weights until we had met the demands of the operations (i.e. found three cards to pass or one card to play). This approach works reasonably well for these operations, though it places heavy responsibility on the weightings, since the system is attempting to make the best choice on the first query match. Human experts may often consider various alternatives in turn. In cases where the human is still using a largely heuristic approach, this generate and test style paradigm focuses on pertinent rules and thus prunes the rule space. We have not yet considered how to incorporate this type of reasoning into our system.

We used a different reasoning paradigm for opponent modelling: we fired all the rules which match the situation to accumulate evidence for beliefs about opponent's strategies. This seems to be a very accurate model of what a human expert does, especially since we use so-called personality objects to represent the various thresholds that the Expert Player needs to cross before reacting.

Even though, our expert system performs its task well, it is sharply limited by its shallowness of understanding (like almost every other expert system). Experts can use their understanding of the Clowns and the Conservatives to thoroughly thrash them. For example, we can shoot against the Conservatives easily, because we know that they hate taking tricks, so we can establish our strong suits by leading moderate strength winners before using our real strength. The Expert Player, however, doesn't have a deep enough understanding of the game - particularly, the whys of its heuristics - or the general inference mechanisms to use this kind of information to modify its behavior accordingly.

5. Extensions

We feel that we could build an excellent player without altering our current architecture. So it seems that, a completely heuristic rule approach is adequate to do pretty well even though we have isolated a wide range of circumstances in which humans seem to be doing something completely different. This is mainly because

Hearts is simple enough to allow a collection of rules to simulate the various skills absolutely needed (e.g. HEARTS shoots without an explicitly represented plan). Now, whether or not we are right about the feasibility of achieving competence using rules, it would be interesting to explore some of the issues raised by incorporating other paradigms. In any case, other more demanding games (e.g. bridge) would certainly require greater sophistication.

The primary issue seems to be how to choose what to do from the wide variety of available options - in other words, control. Representing this control knowledge as rules would provide a flexible environment for discovering effective control strategies. These meta-rules would need a meta-vocabulary. Premises might include the same type of terms as ordinary rules as well as statements about currently active control strategies. Action terms would dictate which rule packets or which other components (search, planning, etc) would be activated.

Research on building a system which has a deeper understanding is currently outside the domain of expert system technology. First steps would be developing a frame-based vocabulary for games in general, and more specifically card games and still more so Hearts. This would lead to more powerful rules and open up the possibility of learning new heuristics as well as mapping heuristics from one game into another (analogy). It would be very interesting to try build a program which learned how to play Spades on its own. Lots of work there. Why exactly did AI give up games?